

## Chipchat\_report\_zw4892

### 1.ExampleA\_binaryToBCD

Iteration1:

```
❏ |cd binary_to_bcd/ && iverilog -g2012 -o binary_to_bcd.vvp binary_to_bcd.v binary_to_bcd_tb.v && vvp binary_to_bcd.vvp
*** binary_to_bcd.v:112: warning: macro binary_to_bcd undefined (and assumed null) at this point.
binary_to_bcd_tb.v: No such file or directory
binary_to_bcd.v:111: syntax error
I give up.
```

Did

Iteration2:

```
❏ |cd binary_to_bcd/ && iverilog -g2012 -o binary_to_bcd.vvp binary_to_bcd.v binary_to_bcd_tb.v && vvp binary_to_bcd.vvp
*** binary_to_bcd.v:112: warning: macro binary_to_bcd undefined (and assumed null) at this point.
binary_to_bcd.v:111: syntax error
I give up.
```

Iteration3:

```
❏ |cd binary_to_bcd/ && iverilog -g2012 -o binary_to_bcd.vvp binary_to_bcd.v binary_to_bcd_tb.v && vvp binary_to_bcd.vvp
*** Input: 0, BCD: x x x
Input: 123, BCD: x x x
Input: 255, BCD: x x x
```

Iteration4:

The same as Iteration3, thought it might be the code wasn't renew.

Use instruction to check the content.

It turns out that the code was updated but the outcome still wrong.

Iteration5:

AI seems like deaf as it didn't solve the problem and didn't initialize the reg.

Iteration6:

AI used sequential logic to generate but we don't have a clock to control the whole system so let it retry again.

Iteration7:

Change the testbench to match the sequential logic that AI provide but Ai 's code is so complicated that have a lot logic mistakes. It turns out always '000'

```
— file renew , start simulate —  
Input: 123, BCD: 0 0 0  
Input: 255, BCD: 0 0 0
```

Iteration8:

It seems like AI has a history of communication with me and It can't get rid of the sequential logic. So I'll directly change the first prompt.

```
new_prompt = '''  
start with this  
`module binary_to_bcd(  
    input [7:0] bin,  
    output reg [3:0] hundreds,  
    output reg [3:0] tens,  
    output reg [3:0] ones  
);  
  
    integer i;  
    reg [19:0] z;  
    ...  
`use simple combinitial logic  
NO clk or res or res_n!!!  
'''
```

```
*** Certainly! Below is a synthesizable Verilog module that implements an 8-bit binary to BCD converter using the double-dabble algorithm. The module takes an 8-bit binary input a  
  
### Verilog Code  
`verilog  
module binary_to_bcd(  
    input wire clk,           // Clock input  
    input wire rst_n,         // Asynchronous reset (active low)  
    input wire [7:0] bin,     // 8-bit binary input  
    input wire start,         // Start signal for conversion  
    output reg [3:0] hundreds, // Hundreds place BCD output  
    output reg [3:0] tens,    // Tens place BCD output  
    output reg [3:0] ones,    // Ones place BCD output  
    output reg done,         // Done signal when conversion completes  
);  
  
    reg [7:0] shift_reg;      // Shift register for binary input  
    reg [3:0] bcd[2:0];       // BCD outputs (hundreds, tens, ones)  
    reg [3:0] count;          // Counter for shifting operations  
    reg state;                // State variable  
  
    localparam IDLE = 1'b0, CONVERT = 1'b1;  
  
    always @(posedge clk or negedge rst_n) begin  
        if (!rst_n) begin  
            // Reset all outputs  
            hundreds <= 4'b0000;  
            tens <= 4'b0000;  
            ones <= 4'b0000;  
            done <= 1'b0;  
            state <= IDLE;  
        end  
        else begin  
            if (state == IDLE) begin  
                // Initialize shift register and counters  
                shift_reg <= bin;  
                count <= 0;  
                state <= CONVERT;  
            end  
            else if (state == CONVERT) begin  
                // Double-dabble algorithm logic  
                // Shift right by one bit  
                shift_reg <= {shift_reg[7:1], 1'b0};  
                // Add 5 to the appropriate BCD digit  
                case (count)  
                    0: hundreds <= hundreds + 5;  
                    1: tens <= tens + 5;  
                    2: ones <= ones + 5;  
                end  
                // Increment counter  
                count <= count + 1;  
                // Check if counter is 3 (all digits processed)  
                if (count == 3) begin  
                    state <= IDLE;  
                    done <= 1'b1;  
                end  
            end  
        end  
    end  
endmodule
```

Iteration9:

```
verilog_generation_prompt = '''
Task: Complete the Verilog module for an 8-bit binary to BCD converter.
Strict Requirements:
1. PURE COMBINATIONAL LOGIC ONLY.
2. NO 'clk', NO 'rst', NO 'reset'.
3. Use 'always @(*)' with blocking assignments (=).

Start with this exact header:
module binary_to_bcd(
    input [7:0] bin,
    output reg [3:0] hundreds,
    output reg [3:0] tens,
    output reg [3:0] ones
);
    integer i;
    reg [19:0] z;

    always @(*) begin
        z = {12'b0, bin};
        // AI: FILL IN THE DOUBLE DABBLE FOR-LOOP HERE
    end
endmodule
'''
```

AI finally use combinational logic.

But in the for loop it made mistake.

It shift left first then check and +3,so the outcome is wrong.

```
// Double Dabble algorithm for converting binary to BCD
for (i = 0; i < 8; i = i + 1) begin
    // Shift left by one
    z = {z[18:0], 1'b0}; // Shift the entire 20-bit register left

    // If the tens or hundreds are 5 or greater, add 3
    if (z[15:12] >= 4'b0101) begin
        z[15:12] = z[15:12] + 4'b0011; // Add 3 to hundreds
    end
    if (z[11:8] >= 4'b0101) begin
        z[11:8] = z[11:8] + 4'b0011; // Add 3 to tens
    end
    if (z[7:4] >= 4'b0101) begin
        z[7:4] = z[7:4] + 4'b0011; // Add 3 to ones
    end
end
end

!cd binary_to_bcd/ && iverilog -g2012 -o binary_to_bcd.vvp binary_to_bcd.v binary_to_bcd_tb.v && vvp binary_to_bcd.vvp
... Input: 123, BCD: 0 3 0
    Input: 255, BCD: 1 9 4
```

Iteration10:

AI can't understand the difference of the order of adding and shifting.

Finally use nearly handmade code to move on.

```
new_prompt = '''
Your logic is still failing the simulation because you are shifting BEFORE checking/adding 3.

Rewrite the always @(*) block exactly as follows:

Initialize z = {12'b0, bin};

In the for loop (0 to 7):

Step 1: IF z[11:8] >= 5, then z[11:8] = z[11:8] + 3;

Step 2: IF z[15:12] >= 5, then z[15:12] = z[15:12] + 3;

Step 3: IF z[19:16] >= 5, then z[19:16] = z[19:16] + 3;

Step 4: ONLY AFTER those 3 checks, perform z = z << 1;

AFTER the loop, assign outputs: hundreds = z[19:16]; tens = z[15:12]; ones = z[11:8];
'''
```

```
# 1. final_code
final_code = """
module binary_to_bcd(
    input [7:0] bin,
    output reg [3:0] hundreds,
    output reg [3:0] tens,
    output reg [3:0] ones
);
    integer i;
    reg [19:0] z;

    always @(*) begin
        z = {12'b0, bin};
        for (i = 0; i < 8; i = i + 1) begin
            if (z[11:8] >= 5) z[11:8] = z[11:8] + 3;
            if (z[15:12] >= 5) z[15:12] = z[15:12] + 3;
            if (z[19:16] >= 5) z[19:16] = z[19:16] + 3;

            z = z << 1;
        end
        hundreds = z[19:16];
        tens = z[15:12];
        ones = z[11:8];
    end
endmodule
"""

# 2. file in
import os
os.makedirs('binary_to_bcd', exist_ok=True)
with open("binary_to_bcd/binary_to_bcd.v", "w") as f:
    f.write(final_code)

# 3. print to verify
print("— verify —")
!cat binary_to_bcd/binary_to_bcd.v
```

```
!cd binary_to_bcd/ && iverilog -g2012 -o binary_to_bcd.vvp binary_to_bcd.v binary_to_bcd_tb.v && vvp binary_to_bcd.vvp
... Input: 123, BCD: 1 2 3
Input: 255, BCD: 2 5 5
```

## 2.Example\_B\_sequencedetector:

Iteration1:

```
verilog_generation_prompt = """
Please design a Verilog module sequence_detector to detect a specific sequence of 3-bit values.

Specifications:

Clock & Reset: Use clk and an asynchronous active-low reset (rst_n).

Input: A 3-bit wire data_in. An enabled signal must be HIGH for detection to work.

Output: sequence_found (1-bit), which stays HIGH for one clock cycle after the sequence is completed.

The Target Sequence: The module must detect these eight 3-bit values in this exact order: 3'b001 -> 3'b101 -> 3'b110 -> 3'b000 -> 3'b110 -> 3'b110 -> 3'b011 -> 3'b101.

Design Style: Use a Moore-type Finite State Machine (FSM) with the three-always-block style (state register, next state logic, output logic).

Use localparam to define states clearly (e.g., S_START, S_VAL1, S_VAL2...).

Provide ONLY the Verilog code.
"""
```

```
... VCD info: dumpfile sequence.vcd opened for output.
start sequence detect simulation...
Time=36000 | Input=001 | State=0
Time=46000 | Input=101 | State=1
Time=56000 | Input=110 | State=2
Time=66000 | Input=000 | State=3
Time=76000 | Input=110 | State=4
Time=86000 | Input=110 | State=5
Time=96000 | Input=011 | State=6
Time=106000 | Input=101 | State=7
failed!: didn't detect the target sequence!
```

We can see that AI's code can successfully jump to correct state but fail to detect the sequence. Because the output was taken in sequence logic so the sequence\_found have a period delay so the testbench completed the sampling before the signal valid.

Iteration2:

```
Your current FSM has an output delay issue. The sequence_found signal is updated inside an always @(posedge clk) block, which introduces a 1-cycle delay and causes my testbench to fail.

Please fix the code with these EXACT changes:

Output Logic: Rewrite the sequence_found logic using a combinational always @(*) block.

Logic Condition: The signal should be 1 when (state == S_VAL7 && enable && data_in == 3'b101). Otherwise, it should be 0.

No Delay: This ensures the output triggers immediately upon receiving the final 3-bit value.

Show me the corrected always @(*) output block
```

```
sequence_found <= (state_next == S_VAL7); // Set found signal when reaching last state
```

AI still puts the final state match code in the always(@posedge..) so the outcome still miss a period

```

*** VCD info: dumpfile sequence.vcd opened for output.
start sequence detect simulation...
Time=36000 | Input=001 | State=0
Time=46000 | Input=101 | State=1
Time=56000 | Input=110 | State=2
Time=66000 | Input=000 | State=3
Time=76000 | Input=110 | State=4
Time=86000 | Input=110 | State=5
Time=96000 | Input=011 | State=6
Time=106000 | Input=101 | State=7
failed!: didn't detect the target sequence!

```

Iteration3:

Change the code and make the final sequence\_found in an assign block in order to escape the one period delay.

```

end
assign sequence_found = (current_state == S_7 && enable && data_in == 3'b101);

```

```

always @(*) begin
    next_state = S_IDLE;
    case (current_state)
        S_IDLE: next_state = (data_in == 3'b001) ? S_1 : S_IDLE;
        S_1: next_state = (data_in == 3'b101) ? S_2 : (data_in == 3'b001 ? S_1 : S_IDLE);
        S_2: next_state = (data_in == 3'b110) ? S_3 : (data_in == 3'b001 ? S_1 : S_IDLE);
        S_3: next_state = (data_in == 3'b000) ? S_4 : (data_in == 3'b001 ? S_1 : S_IDLE);
        S_4: next_state = (data_in == 3'b110) ? S_5 : (data_in == 3'b001 ? S_1 : S_IDLE);
        S_5: next_state = (data_in == 3'b110) ? S_6 : (data_in == 3'b001 ? S_1 : S_IDLE);
        S_6: next_state = (data_in == 3'b011) ? S_7 : (data_in == 3'b001 ? S_1 : S_IDLE);
        S_7: next_state = (data_in == 3'b101) ? S_IDLE : (data_in == 3'b001 ? S_1 : S_IDLE);
        default: next_state = S_IDLE;
    endcase
end
// when at S_7 and the eighth is 101 , immediately get a high voltage
assign sequence_found = (current_state == S_7 && enable && data_in == 3'b101);

```

```

*** --- compiling and simulating... ---
--- start test sequence: 001-101-110-000-110-110-011-101 ---
Time=36000 | Input=001 | State=0
Time=46000 | Input=101 | State=1
Time=56000 | Input=110 | State=2
Time=66000 | Input=000 | State=3
Time=76000 | Input=110 | State=4
Time=86000 | Input=110 | State=5
Time=96000 | Input=011 | State=6
SUCCESS: Sequence Detected!

```

Part III:

Extend design: 4 bit up down counter:

Promts:

```

Please design a 4-bit Up/Down Counter in Verilog. Requirements:

Ports: clk, rst_n (async active low), en (enable), up_down (1: up, 0: down), output count [3:0], and output overflow.

Behavior: The counter increments if up_down is 1 and decrements if it is 0, but only when en is high.

Reset: On rst_n = 0, the count should be 0.

Overflow Logic: Use an assign statement (combinational logic) for overflow. overflow should be 1 if (count == 15 and up_down == 1) or (count == 0 and up_down == 0).

Ensure the count is updated in a sequential always @(posedge clk or negedge rst_n) block.
***

```

Test\_bench:

```

▶ tb_code = """
`timescale 1ns / 1ps
module up_down_counter_tb;
    reg clk, rst_n, en, up_down;
    wire [3:0] count;
    wire overflow;

    up_down_counter uut (
        .clk(clk), .rst_n(rst_n), .en(en), .up_down(up_down),
        .count(count), .overflow(overflow)
    );

    always #5 clk = ~clk;

    initial begin
        clk = 0; rst_n = 0; en = 0; up_down = 1;
        #20 rst_n = 1; #10 en = 1;

        // up till out
        repeat(16) @(posedge clk);

        // down
        @(posedge clk); #1 up_down = 0;
        repeat(5) @(posedge clk);

        $display("Final Count = %d, Overflow = %b", count, overflow);
        if (count == 11) $display(" SUCCESS: Extension Module passed test!");
        else $display(" FAILED: Count mismatch.");

        #20 $finish;
    end
endmodule
***

```

Outcomes:

```

***  — compiling Extension simulation —
Final Count = 11, Overflow = 0
      SUCCESS: Extension Module passed test!

```